

```
In [5]: import pandas as pd
        #importing pandas for high performance data manipulation
```

```
In [7]: dataset = "/home/pict/31436/assignment_1/Iris.csv"
        #copying local dataset in the same folder

        column_names = [
            "SepalLengthCm",
            "SepalWidthCm",
            "PetalLengthCm",
            "PetalWidthCm",
            "Species"
        ]
        #displaying attributes for each element

        df = pd.read_csv(dataset)
        #loading dataset into pandas dataframe
```

```
In [8]: df.head()
        #linux command head used for outputing first 5 entries of dataframe
```

```
Out[8]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [9]: df.dtypes
        #call and check data types of all attributes
```

```
Out[9]: Id                int64
SepalLengthCm          float64
SepalWidthCm           float64
PetalLengthCm          float64
PetalWidthCm           float64
Species                object
dtype: object
```

```
In [10]: df["Species"] = df["Species"].astype('str')
#normalize the attribute to ensure uniformity of all values

df.isnull().sum()
#checks each val if is null and if so, adds it together and presents the sum of all null values
```

```
Out[10]: Id                0
SepalLengthCm            2
SepalWidthCm             2
PetalLengthCm            3
PetalWidthCm             0
Species                  0
dtype: int64
```

```
In [11]: df.describe()
#describes various statistical measures applicable information about the dataset
```

```
Out[11]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	148.000000	148.000000	147.000000	150.000000
mean	75.500000	5.848649	3.053378	3.770748	1.198667
std	43.445368	0.830743	0.436345	1.770272	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.400000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
In [12]: df.info()
#provides information about the dataset such as missing values and datatypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               150 non-null    int64
1   SepalLengthCm    148 non-null    float64
2   SepalWidthCm     148 non-null    float64
3   PetalLengthCm    147 non-null    float64
4   PetalWidthCm     150 non-null    float64
5   Species          150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
In [13]: df.shape
#outputs the size of dataset in (row, column) format
```

```
Out[13]: (150, 6)
```

```
In [14]: mean_SepalLengthCm = df["SepalLengthCm"].mean()
#continuous normally distributed numerical variables with a very small proportion of missing values

df["SepalLengthCm"].fillna(mean_SepalLengthCm , inplace=True)
#checks values in succession if null, and if true, repace the null value with mean_SepalLengthCm

df.info()
#provides information about the dataset such as missing values and datatypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               150 non-null   int64
1   SepalLengthCm    150 non-null   float64
2   SepalWidthCm     148 non-null   float64
3   PetalLengthCm    147 non-null   float64
4   PetalWidthCm     150 non-null   float64
5   Species          150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

/tmp/ipykernel_14183/1382140515.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["SepalLengthCm"].fillna(mean_SepalLengthCm , inplace=True)    #checks values in succession if null, and if true, repac the null value with mean_SepalLengthCm
```

In [15]:

```
mean_SepalWidthCm = df["SepalWidthCm"].mean()
#continuous normally distributed numerical variables with a very small proportion of missing values

df["SepalWidthCm"].fillna(mean_SepalWidthCm , inplace=True)
#checks values in succession if null, and if true, repac the null value with mean_SepalWidthCm

df.info()
#provides information about the dataset such as missing values and datatypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               150 non-null   int64
1   SepalLengthCm    150 non-null   float64
2   SepalWidthCm     150 non-null   float64
3   PetalLengthCm    147 non-null   float64
4   PetalWidthCm     150 non-null   float64
5   Species          150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

/tmp/ipykernel_14183/3138791438.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["SepalWidthCm"].fillna(mean_SepalWidthCm , inplace=True)    #checks values in succession if null, and if true, repa
```

In [16]:

```
median_PetalLengthCm = df["PetalLengthCm"].median()
#continuous skewed data immune to extreme values with a very small proportion of missing values

df["PetalLengthCm"].fillna(median_PetalLengthCm , inplace=True)
#checks values in succession if null, and if true, repa the null value with median_PetalLengthCm

df.info()
#provides information about the dataset such as missing values and datatypes
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               150 non-null   int64
1   SepalLengthCm    150 non-null   float64
2   SepalWidthCm     150 non-null   float64
3   PetalLengthCm    150 non-null   float64
4   PetalWidthCm     150 non-null   float64
5   Species          150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

/tmp/ipykernel_14183/1184438810.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["PetalLengthCm"].fillna(median_PetalLengthCm , inplace=True)      #checks values in succession if null, and
if true, repace the null value with median_PetalLengthCm
```

```
In [17]: df.isnull().sum()
#checks each val if is null and if so, adds it together and presents the sum of all null values
```

```
Out[17]: Id                0
SepalLengthCm            0
SepalWidthCm             0
PetalLengthCm            0
PetalWidthCm            0
Species                 0
dtype: int64
```